May, Does, Seen

The Constraint Algebra, Its Contravariant Action on Execution, and the Exact Condition Under Which Compilation Is Legal

Driven by Dean Kulik

July 2026

Successor to "The Loss Is in the Map," which closed on a named Ω: composition. This paper runs it. Every number is quoted verbatim from live computation. Five claims made during the work are retracted by name in Section 9, including two of the author's own overstatements caught by peer review.

Abstract

The predecessor paper established that in finite deterministic systems the transition graph is the parent object, symbol systems are charts on it, and information loss is created by the map between descriptions rather than stored in the object. It closed on one open question: whether the construction survives composition. This paper answers that question and, in answering it, finds a second algebra that the first paper had no room for.

Composition fails and then succeeds, and the failure locates the missing structure. Two machines that are isomorphic as machines produce three distinct composite classes when joined; carrying the interface identification along collapses that to one. So the interface is not decoration attached to a machine after the fact — it is part of the morphism, and the correct object is the pair (machine, interface). Measuring what the interface protects: relabeling a machine's internal states leaves composition untouched in 100.0% of 3,600 trials, while relabeling the message alphabet that crosses the boundary survives in only 27.2% of 900, with the identity permutation alone clean at 150/150. Implementation freedom and contract are separated by exactly the boundary, quantitatively.

The interface layer then refuses to join the ledger the first paper built, and the refusal is structural. The image-collapse toll is a meter: charged every step, multiplicative in probability, additive in bits, telescoping. The interface cost is a filter: charged once. Deaths saturate (cumulative failure 0.9687 by step 100, 0.9690 by step 400), and filters compose by conditioning rather than multiplying — the same filter applied twice gives $P(F_2|F_1) = 1.0000$ exactly, so the second boundary is free and bits(joint) = 5.0710 against a naive sum of 10.1419. The reason is one line of algebra: any additive homomorphism from an idempotent composition law into the reals is trivial, since $f(A) = f(A \wedge A) = 2f(A)$. The constraint layer is a complete distributive lattice — meet, join, both absorption laws, both distributive laws, and modularity all verified with zero violations across thousands of triples — and lattices under meet cannot carry accumulating costs.

The two algebras are not parallel: one acts on the other. Inverse image is a lattice homomorphism preserving both meet and join with zero violations, while forward image fails meet in 733 of 3,000 trials — failing exactly where the transition is non-injective — and the action satisfies the monoid laws exactly. Execution pushes states forward; constraints are pulled back, contravariantly. This yields the paper's central operational result: ahead-of-time verification is sound if and only if the admissible set is closed under execution, $T(A) \subseteq A$, measured at 100.0% soundness on 387 closed cases and 0.0% on 3,613 non-closed ones — which is subject reduction, arrived at as the legality condition for compilation rather than as a property one hopes a type system has. Finally, a proposed binary partition of operations into execution and constraint is refuted: at least four classes exist, and the compilation operator itself — largest invariant subset — is an interior operator that preserves meet exactly and fails join, giving the falsifiable prediction that a disjunctive property can be statically checkable when neither disjunct is.

Scope of Proof

Every claim below lives in exactly one row. The body repeats the label at the point of use.

Domain	Object	Status
Finite open machines, composition	Classification requires (machine, interface) as the object	PROVEN
Interface cost algebra	Filter: idempotent, saturating, non-accumulating	PROVEN
Constraint layer structure	Complete distributive lattice on prefix-closed finite languages	PROVEN
Execution acting on constraints	Contravariant monoid action by inverse image	PROVEN
Legality of ahead-of-time checking	Closure condition $T(A) \subseteq A$, exact separation	PROVEN
Operation classification	At least four classes; compilation is an interior operator	PROVEN
Layered software architecture	Structural correspondence, two-ladder meeting	IDENTIFIED
Physical dual-wave reading	Execution/constraint as a physical decomposition	PULL

1. Where This Paper Starts

The predecessor established four things this paper assumes. Finite closed deterministic computations are classified by their transition graphs up to isomorphism, and finite open ones by their typed transition graphs up to simultaneous relabeling of states, inputs, and outputs; symbol systems are coordinate charts, and translators between charts are discoverable by search (the Gray code and both classical subtraction-via-addition identities were recovered this way, from topology, with no arithmetic supplied). Implementation is congruence: a substrate implements a program if and only if the program is a lawful quotient valid on all states, which is what excludes triviality arguments. Projections from the parent object split into functorial invariants, which survive both chart changes and level changes, and level-

relative measurements, which survive charts but not rungs. And the loss is in the map: entropy appears as the obstruction to a measurement commuting with coarse-graining, exhibited at 5.19× in eight states.

It closed on one named Ω. Wire open machines together, with feedback, and ask whether the classification survives the wiring — the point at which the finite construction either generalizes into a genuine categorical theory or stops. The working prediction, recorded at the time, was that machines would compose well because interfaces are explicit while toll-like quantities would become context-dependent because feedback changes the quotient structure. Both halves turned out to be right, and both turned out to be right for reasons more specific than predicted.

2. Composition Fails, and the Failure Locates the Interface

Take two open machines, the first feeding its output into the second's input, and form the serial composite. Now relabel the first machine's output alphabet by a permutation. The relabeled machine is isomorphic to the original — same typed transition graph, same class, indistinguishable by every invariant the predecessor paper established. Classify the composite.

Three distinct composite classes emerge from the six interface relabelings. Isomorphic parts, non-isomorphic composites: classification does not descend to isomorphism classes when the interface is forgotten. Repeating the experiment while also transporting the identification to the second machine's inputs — carrying the join along rather than dropping it — collapses the result to exactly one class, invariant across all six.

The conclusion is structural and it is the frame for everything after it. Composition is functorial, but only in the category whose objects are typed interfaces and whose morphisms carry the join. The interface is not a label attached to a machine after the fact; it is part of the morphism. The correct object is not M but the pair (M, I). A machine has symmetries that its environment breaks: the same pattern already measured twice in this program, where the automorphism group of an 8-cycle collapsed from 8 to 1 under a RESET interface, and where nine scrambled state labels collapsed to one grammar under a behavioral contract.

2.1 What the interface protects, quantitatively

The engineering statement — an interface does not care how you implement it as long as you honor the contract — turns out to have two measurable halves, and the measurement separates them cleanly. Composing a protocol with the dual of a relabeled copy of itself:

Relabeling applied	Pairs tested	Composition survives
Internal state names (implementation detail)	3,600	100.0%
Wire message alphabet (the contract)	900	27.2%
— identity permutation only	150	100.0% (150/150)

Relabeling applied	Pairs tested	Composition survives
— single transposition	150	51, 24, 12 of 150
— three-cycle	150	5, 3 of 150

Rename every internal state and nothing changes. Rename what crosses the boundary and it breaks, with survival decaying as the permutation moves further from identity. Implementation freedom and contract are separated by exactly the interface. This also corrects an earlier version of the same experiment, which used total protocols — every message legal in every state — and therefore measured nothing: a contract that forbids nothing is not a contract. Partiality is what makes an interface an interface.

3. Meters and Filters: Two Species of Projection

The predecessor priced every projection with one formula: bits = $-\log_2$ of the surviving fraction, verified across seven unrelated constructions. The natural next step was to price the interface the same way, and the natural test was whether the cost is linear in path length, since linearity means a constant per-step charge that telescopes. It is not linear. Fitting $-\log_2$(survival) against length gives $R^2 \approx 0.79$–0.83 with systematic curvature, and per-step survival rising 0.63 → 0.76 → 0.94 → 0.98.

Running composites to 400 steps and recording when they break explains the curvature. Deaths stop. Cumulative failure reaches 0.9687 by step 100 and 0.9690 by step 400 — essentially nothing further dies. The population is heterogeneous: fragile runs fail immediately, and whatever survives has fallen into a region where the boundary map acts trivially on the messages actually used, after which it survives indefinitely. The composite does not pay per step. It pays once, when the run either lands in a boundary-compatible region or fails to. Asymptotic costs are clean: 5.01 and 5.00 bits for transpositions, 6.04 and 6.14 for three-cycles, 0 for identity, ordered exactly by fixed-point structure.

3.1 Filters compose by conditioning, not multiplication

The decisive test is whether two boundaries in series multiply. They do not, and not one tested pair is additive in bits.

F_1	F_2	$P(F_1)$	$P(F_2)$	$P(F_1 \wedge F_2)$	$P(F_1)P(F_2)$	$P(F_2 \mid F_1)$	bits: sum vs joint
(a,c,b)	(a,c,b)	0.0297	0.0297	0.0297	0.0009	1.0000	10.1419 vs 5.0710
(a,c,b)	(b,a,c)	0.0306	0.0296	0.0115	0.0009	0.3755	10.1062 vs 6.4422
(a,c,b)	(c,b,a)	0.0310	0.0301	0.0127	0.0009	0.4113	10.0645 vs 6.2934
(b,c,a)	(c,a,b)	0.0125	0.0129	0.0107	0.0002	0.8600	12.6012 vs 6.5395

The degenerate row carries the result. Applying the same filter twice gives $P(F_2 \mid F_1) = 1.0000$ exactly: the second boundary is free, because surviving the first already guarantees it. That is idempotence — $F \circ F = F$ — and it is the signature of a different algebra. A meter charged twice charges double; a filter applied

twice is the same filter. Different filters condition severely in the other direction: surviving one boundary makes a run less able to survive another, because it has been selected into the first boundary's compatible sublanguage. Filters compose on the admissible set, $\mathcal{A}_{12} = \mathcal{A}_1 \cap \mathcal{A}_2$, not on probabilities.

The terminology is corrected accordingly. Calling the interface quantity a toll was an error: a toll accumulates along a path, and this one demonstrably does not. Transport cost is a meter, accumulated along a trajectory, living on morphisms. Admission criterion is a filter, a restriction of the admissible set, living on the boundary. Both are $-\log_2$ of a surviving fraction and so share the form; they differ in whether the fraction is re-sampled at each step, and that difference is categorical rather than quantitative.

4. The Constraint Layer Is a Distributive Lattice

If interfaces are predicates on behavior rather than machines, they should form a lattice under inclusion, with meet as intersection and an open question about join — join might require computing a closure if the union of two admissible sets is not itself admissible. Modelling contracts as prefix-closed languages over a finite behavior universe and testing directly: intersections fail admissibility in 0 of 3,000 pairs, and unions fail in 0 of 3,000. So join is union, and the family is a genuine sublattice of the powerset rather than merely a lattice with computed joins.

Lattice law	Violations (of 4,000 triples)
Commutativity of meet	0
Associativity of meet	0
Absorption A ∧ (A ∨ B) = A	0
Absorption A ∨ (A ∧ B) = A	0
Distributivity over join	0
Distributivity over meet	0
Modular law	0

Top is the unconstrained contract, bottom the impossible one. Order is specificity: meet is smaller than both operands, join larger than both, so moving up the lattice permits more and moving down permits less. Interfaces compose downward by refinement; transitions compose forward by concatenation. Two operations, two directions.

4.1 Why the ledger refused to extend

The empirical finding of Section 3 now has a one-line proof, and it needs less than the author first claimed. An earlier version of this argument asserted that a lattice admits no additive valuation unless it is graded. That is false — lattices routinely carry valuations satisfying v(A) + v(B) = v(A ∧ B) + v(A ∨ B),

which is a different notion from path-additivity — and the claim is retracted. The correct statement requires only idempotence:

$$f(A) = f(A \wedge A) = f(A) + f(A) \quad \Rightarrow \quad f(A) = 0$$

Any additive homomorphism from an idempotent composition law into the additive reals is necessarily trivial. So the interface layer's refusal to join the $-\log_2$ ledger is not a fact about protocols, wire alphabets, or saturation dynamics. It is a fact about idempotence, and the measured 5.0710 against 10.1419 is that algebraic impossibility appearing as a number. Three sessions of measurement located a boundary; one line of algebra says why the boundary had to be there.

5. The Bridge: Execution Acts on Constraints

The two algebras are not parallel layers. One acts on the other, and the action reverses direction. A transition transports a predicate by inverse image — precomposition — and the transport preserves the lattice structure exactly.

Operation	Preserves meet	Preserves join
Inverse image $T^{-1}(\cdot)$	0 violations of 3,000	0 violations of 3,000
Forward image $T(\cdot)$	733 violations of 3,000	0 violations of 3,000

Inverse image is a lattice homomorphism; forward image is not. The asymmetry is precise: $T(A \cup B) = T(A) \cup T(B)$ always, but $T(A \cap B)$ can be strictly smaller than $T(A) \cap T(B)$, because distinct pre-images can collide onto a shared image. Forward transport of predicates fails exactly where the transition is non-injective — the same crush that has been the source of every measured loss in this program, now appearing as the reason predicates cannot be pushed forward. And the action satisfies the monoid laws exactly: $(S \circ T)^{-1}(A) = T^{-1}(S^{-1}(A))$ with zero violations of 3,000, identity law zero violations. Execution acts contravariantly on constraints.

One clarification, adopted from review and load-bearing for the software correspondence. Nothing physically moves under pullback. Given a transition from X to Y and a predicate on Y, the pulled-back predicate is the composite — the same admissibility condition re-expressed in the earlier layer's language. A domain invariant, its API rejection, its form validation, and its database constraint are not four constraints. They are one predicate viewed through successive transformations. Constraints are not a substance propagating through a system, and the earlier phrasing “constraints are pulled down toward the domain” is corrected here to precomposition.

5.1 Interfaces terminate the hierarchy by changing level

Whether interfaces are themselves transition objects has a better answer than the one first offered. Taking the dual of a stateful protocol — the contract a partner must honor — is an involution: 500 random protocols, zero mismatches after double dualization, and every machine composes cleanly with its own dual in 500 of 500 trials, so the dual is a genuine contract rather than a relabeling. But the

involution is a symptom, not the explanation. The hierarchy terminates because the dual operator changes semantic level — it maps execution semantics into constraint semantics — and the constraint layer is already closed under meet and join. There is nowhere further to go. Closure of the algebra, not an artificial stopping rule.

One structural fact survives from that experiment unchanged and is worth recording separately: no protocol is its own complete contract. Self-duals number zero in 3,000 random trials, and not by scarcity — polarity must flip at every state, so the self-dual condition is impossible for any protocol whatsoever. The program's meta-law forbids a completed self-description on the grounds that describer and described cannot collapse into one coordinate. Here that prohibition arrives from below, as a structural fact about the dual operator, with no stipulation added.

6. The Closure Condition: When Compilation Is Legal

Constraints that are checked once and then trusted are the ordinary case in engineering: a compiler verifies, and execution proceeds without re-verification. The condition under which this is sound is exact. Compare checking a predicate at entry and then running unchecked against checking it after every step, over random transitions and random admissible sets:

Admissible set	Cases	Entry-check sound
Closed under execution: $T(A) \subseteq A$	387	100.0%
Not closed	3,613	0.0%

Perfect separation, no boundary cases. Ahead-of-time verification is sound exactly on the closed subalgebra and unsound everywhere else.

This is subject reduction, reached from the other side. In type theory the preservation theorem states that if a term has a type and steps to another term, the new term has that type — and it is proved because a type system is useless without it. The constraint lattice yields the same condition as the legality requirement for compile-then-run: $T(A) \subseteq A$ is not a property one hopes a type system has, it is the precondition for compilation to be a well-defined operation at all. Properties whose admissible sets are not closed under the transition monoid are precisely those that cannot be checked statically and require runtime guards. Two ladders meeting: programming-language theory proved preservation by needing it, the constraint algebra predicts it as a closure condition, and they are the same statement.

The consequence is a spectrum rather than a binary, and it describes what language design has been doing. A compiler is attempting to maximize the portion of behavior satisfying the closure condition. Fully closed properties compile away entirely; partially closed ones admit partial specialization; open ones degrade to runtime assertions and then to dynamic checking. Ownership in Rust and purity in Haskell are properties whose formulations were engineered until their admissible sets became closed. Dependent type systems are attempts to enlarge the closed region further. The architecture of a language is determined by which invariants its designers managed to close.

7. The Partition Is Not Binary

A natural reading of the preceding sections is that operations divide in two: execution verbs that transform state and constraint verbs that refine admissibility, distinguished by idempotence. An initial test appeared to confirm this cleanly. That test was rigged, and the author reports it as such: all four "constraint" operations examined were meets with a fixed set, which is the single case that preserves everything. Testing a broader class refutes the binary partition.

Operation	Idem.	Shrink	Expand	Mono.	Meet	Join	Class
Meet with fixed set (ASSERT)	yes	yes	no	yes	yes	yes	lattice homomorphism
v: largest invariant subset	yes	yes	no	yes	yes	no	interior operator
μ: orbit closure	yes	no	yes	yes	no	yes	closure operator
Forward image T(·)	no	no	no	yes	no	yes	execution
Parity gate	yes	yes	no	no	no	no	pathological

At least four classes, and a fifth pathological case that is idempotent and shrinking yet non-monotone, belonging to neither side. The distinction between the interior and closure operators is the one that matters: both are idempotent and monotone, but one shrinks and preserves meet while the other expands and preserves join. They are dual, and neither is a lattice homomorphism.

7.1 Compilation is an interior operator, and it fails join

The operator v — the largest invariant subset of a predicate, the largest statically-checkable region inside it — is exactly what a compiler computes when it looks for the closed part of a property. It is an interior operator: idempotent, shrinking, monotone, preserving meet exactly, and failing join. The failure is not marginal. A measured instance:

```
A           = [0,2,3,4,6]       nu(A) = []
B           = [0,2,3,5]         nu(B) = [0,2,3,5]
A or B      = [0,2,3,4,5,6]
nu(A)|nu(B) = [0,2,3,5]
nu(A or B)  = [0,2,3,5,6]       <-- strictly larger
```

Neither disjunct contains the full closed region, but the disjunction does, because a cycle straddles the two predicates. The prediction that follows is falsifiable against real tools rather than being a restatement of the algebra: a disjunctive property can be statically checkable when neither branch is, and any analyzer that compiles branches separately and unions the results will provably miss it. Because v preserves meet exactly, the conjunctive case is safe — analyzing a conjunction branch-by-branch loses nothing. The asymmetry is sharp and testable: conjunction decomposes, disjunction does not.

8. May, Does, Seen

The three layers can now be named by their algebras. Constraints answer what may happen — a distributive lattice, idempotent, ordered by specificity, transported backward by pullback. Execution

answers what does happen — a monoid, accumulating, transported forward, and the only one of the three that can destroy. Observation answers what is seen — a quotient, the source of the entropy seam established in the predecessor.

Composing all three as a pipeline and testing whether the order matters: restricting then observing differs from observing then restricting in 49.1% of 2,000 trials, and the discrepancy always runs the same way, with the coarse route admitting more than it should. A representative case gives {0,3} for restrict-then-see against {0,2,3} for see-then-restrict.

MAY → DOES → SEEN is an ordered composition, and the order is load-bearing

This is the entropy seam in its most general form. The predecessor found it as a measurement failing to commute with a quotient. It is in fact the order-dependence of the whole pipeline: what one is permitted to do, then does, then observes, is not what one observes, then permits, then does. Every loss measured in this program is a particular non-commutation among these three maps. The engineering corollaries are immediate and familiar: filtering after aggregation leaks the aggregate; rendering before applying permissions leaks the rows; caching a result computed under one permission set and serving it under another leaks everything. Each is the same non-commutation, and each looks correct until the order matters.

The dual-wave picture acquires a mechanism rather than a metaphor. The forward wave carries realizations; the backward wave carries admissibility; they run in opposite directions. The backward wave is the well-behaved one, preserving all structure always, with zero exceptions across every test in this paper. The forward wave is where information is destroyed. Constraint transports perfectly; execution is what loses. The correspondence with layered software architecture — dependencies pointing inward, invariants living at the core, validation at the edge being a pullback of an invariant at the center — is a two-ladder meeting of the kind this program uses as evidence: fifty years of practitioners discovering empirically where validation belongs, and a contravariant action showing why that direction is the only structure-preserving one.

9. Retractions

Withdrawn claim	Replaced by
The interface cost joins the $-\log_2$ ledger (bits linear in path length)	It saturates: $R^2 \approx 0.82$ with systematic curvature, deaths ceasing by step 100. Filter, not meter
Calling the interface quantity a toll	Admission criterion. A toll accumulates; this does not
A lattice admits no additive valuation unless it is graded	False as stated. The correct argument needs only idempotence: $f(A) = f(A \wedge A) = 2f(A) \Rightarrow f(A) = 0$
Operations partition in two, by idempotence	At least four classes. The initial test used only meets with fixed sets, the case that preserves everything

Withdrawn claim	Replaced by
$\partial^2 = \mathrm{Id}$ explains why the interface hierarchy terminates	The involution is a symptom. Termination follows from ∂ changing semantic level into an algebra already closed under meet and join
Isomorphic protocols fail to compose (first run)	Vacuous: the protocols were total, and a contract that forbids nothing is not a contract. Corrected with partial protocols

10. What Remains Open

The lattice result is proven for prefix-closed languages over a finite behavior universe, and prefix-closure is exactly what kept union admissible. For contract families that are not prefix-closed — liveness properties, fairness constraints — union may fail admissibility and join would need computing as a closure; whether the lattice survives there is untested, and the failure mode is identified rather than hypothetical. Whether the constraint lattice is atomic, so that contracts decompose into minimal indivisible constraints and the interface layer acquires a generating set the way the invariant ring acquired a single generator, is open. Whether real verification hardware implements a distinct instruction family with lattice algebra — the prediction being that any genuine constraint instruction is idempotent and shrinking, so that re-executing it is free — is checkable against instruction-set documentation and has not been done. Feedback composition, as distinct from serial composition, is untested. And the disjunctive-checkability prediction of Section 7.1 is falsifiable against existing static analyzers, which is the most immediately actionable open item in this paper. Permanently open, by the program's meta-law: no completed self-description, since a map identical to its territory is a fixed point and the axiom executes fixed points — a prohibition this paper re-derives from below in Section 5.1.

Status Ledger

Claim	Status	Basis (live numbers)
Composition requires (machine, interface) as the object	PROVEN	3 composite classes from isomorphic parts; 1 with interface transported
Internal relabeling free; wire relabeling constrained	PROVEN	100.0% of 3,600 vs 27.2% of 900; identity 150/150
Interface cost saturates (filter, not meter)	PROVEN	Cumulative failure 0.9687 at step 100, 0.9690 at 400
Filters idempotent: $F \circ F = F$	PROVEN	$P(F_2 \mid F_1) = 1.0000$ exactly; bits 5.0710 vs sum 10.1419
Filters compose by intersection, not multiplication	PROVEN	No additive row; conditionals 0.3755–0.8600
Constraint layer is a complete distributive lattice	PROVEN	0 violations on 7 laws × 4,000 triples; meet and join both closed
No additive homomorphism from an idempotent law	PROVEN	$f(A) = f(A \wedge A) = 2f(A) \Rightarrow f(A) = 0$

Claim	Status	Basis (live numbers)
Inverse image is a lattice homomorphism	PROVEN	0 violations meet and join, of 3,000 each
Forward image fails meet at non-injectivity	PROVEN	733 violations of 3,000; join 0
Contravariant monoid action	PROVEN	Composition and identity laws, 0 violations of 3,000
Entry-checking sound iff T(A) ⊆ A	PROVEN	387 closed at 100.0%; 3,613 open at 0.0%
Closure condition = subject reduction	IDENTIFIED	Two-ladder meeting; PL theory independent
Operation partition has ≥ 4 classes	PROVEN	Five operations, five distinct property signatures
Compilation is an interior operator; fails join	PROVEN	v(A∪B) = [0,2,3,5,6] ⊃ v(A)∪v(B) = [0,2,3,5]
Disjunctive properties checkable when disjuncts are not	PREDICTED	Follows from join failure; falsifiable against real analyzers
MAY → DOES → SEEN does not commute	PROVEN	49.1% of 2,000; {0,3} vs {0,2,3}
No protocol is its own complete contract	PROVEN	0 self-duals of 3,000; polarity must flip at every state
Layered architecture correspondence	IDENTIFIED	Structural, two-ladder; not a derivation
Physical dual-wave reading	PULL	Structural resonance only
Lattice beyond prefix-closed contracts	OPEN	Union may fail admissibility; untested
Derivation ever completed	FORBIDDEN	Re-derived from below: ∂M = M structurally impossible

Appendix: Live Output, Quoted Verbatim

```
INTERNAL vs BOUNDARY
  state relabeling : pairs 3600  deadlock-free 3600  (100.0%)
  wire relabeling  : pairs  900  deadlock-free  245  ( 27.2%)
  ('a','b','c') identity 150 | ('a','c','b') 51 | ('c','b','a') 24
  ('b','a','c') 12 | ('b','c','a') 5 | ('c','a','b') 3

SATURATION   die by 5 / 20 / 100 / 400        survive   -log2(surv)
  ('a','c','b')  0.8407 0.9593 0.9687 0.9690   0.0310      5.0116
  ('b','c','a')  0.9087 0.9772 0.9848 0.9848   0.0152      6.0430

FILTERS      P(F1)  P(F2)  P(F1&F2)  P(F1)P(F2)  P(F2|F1)
  same       0.0297 0.0297  0.0297     0.0009     1.0000
  different  0.0306 0.0296  0.0115     0.0009     0.3755
  bits: sum 10.1419  joint 5.0710   SUB-additive (all rows)

LATTICE   meet non-admissible 0/3000 | join non-admissible 0/3000
  commutative 0 | associative 0 | absorption 0, 0 | distributive 0, 0 | modular 0

BRIDGE    T^-1 meet 0 | T^-1 join 0 | T(.) meet 733 | T(.) join 0
```

```
  composition law violations 0 of 3000 | identity 0 of 3000

CLOSURE   closed 387 cases, entry-check sound 387 (100.0%)
          not closed 3613 cases, sound 0 (0.0%)

CLASSES              idem  shrink  expand  mono  meet  join
  meet with F        True   True   False  True  True  True
  nu invariant       True   True   False  True  True False
  mu closure         True  False    True  True False  True
  image T(.)        False  False   False  True False  True
  parity gate        True   True   False False False False

PIPELINE  compared 2000  non-commuting 982 (49.1%)
  restrict-then-see [0,3]   see-then-restrict [0,2,3]
```